

Speaker Script

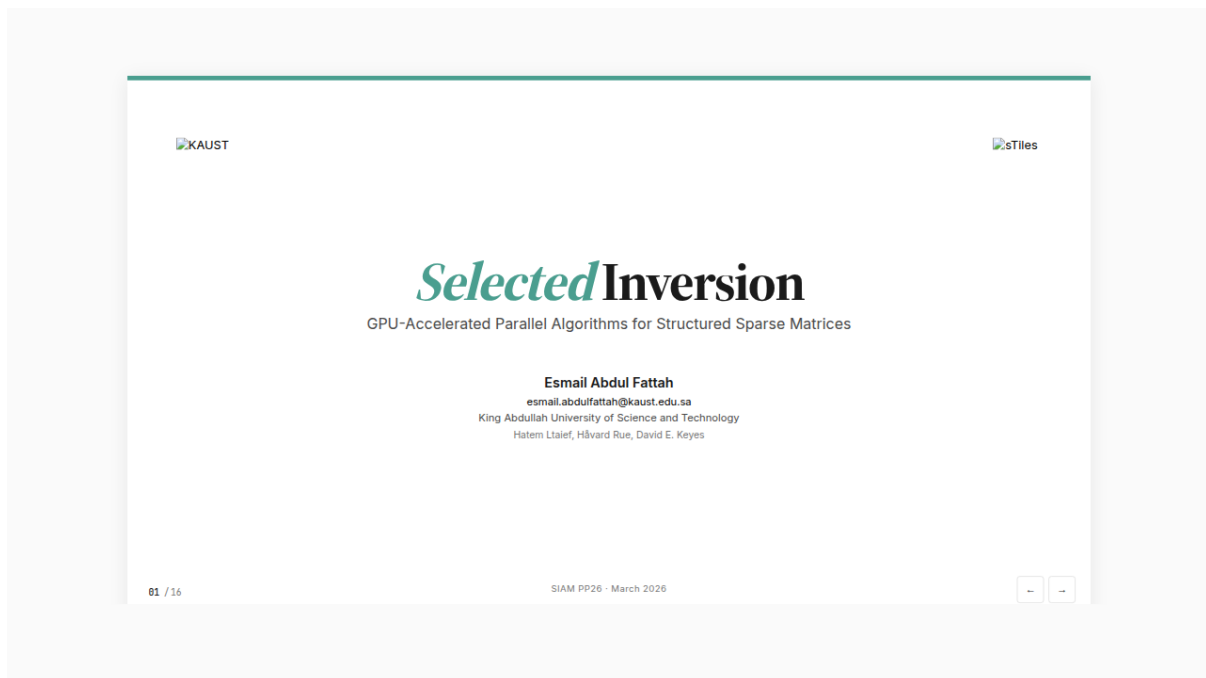
GPU-Accelerated Parallel Selected Inversion for Structured Matrices

SIAM Conference on Parallel Processing (PP26) — March 2026

Total Presentation Time: 20–25 minutes

Style: Professional, clear, technically precise

Slide 1: Title Slide



Speaker Notes

Good morning/afternoon everyone. Thank you for joining this session.

Today I'll be presenting our work on GPU-accelerated parallel selected inversion for structured matrices. This is joint work with my collaborators, and the implementation is part of our sTiles framework.

Selected inversion is a fundamental operation in many scientific computing applications, and we've developed a highly efficient tile-based approach that significantly outperforms existing state-of-the-art solvers.

Slide 2: What is Selected Inversion

INTRODUCTION

What is Selected Inversion?

Computing only the inverse elements that matter

PROBLEM

Given a sparse matrix A , we need specific elements of A^{-1} . However, the full inverse is typically **dense**, making direct computation prohibitive.

$$A \cdot A^{-1} = I$$

KEY IDEA

Compute only $(A^{-1})_{ij}$ where $A_{ij} \neq 0$, preserving the sparsity pattern.

02 / 16

Speaker Notes

Let me start by explaining what selected inversion is and why it matters.

Given a sparse symmetric positive definite matrix A , we often need elements of its inverse A^{-1} . The naive approach would be to compute the full inverse, but this is problematic for two reasons.

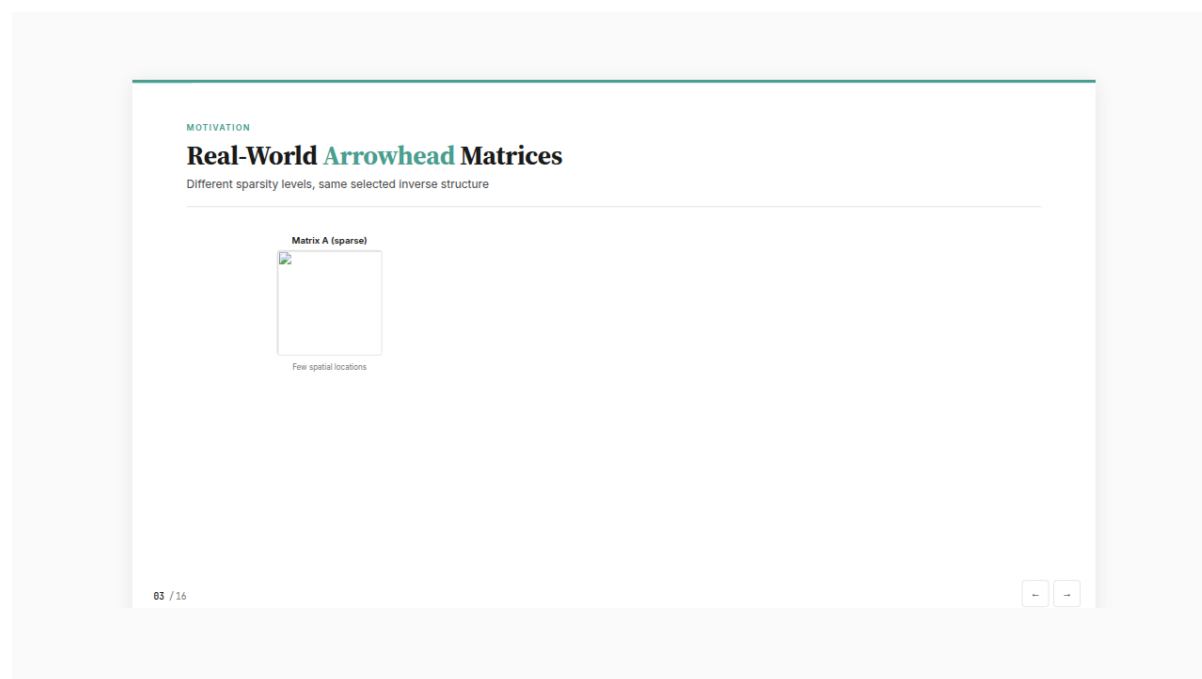
First, even if A is sparse, A^{-1} is generally **dense**. This means we'd need to store and compute n^2 elements instead of just the nonzeros.

Second, computing the full inverse has $O(n^3)$ complexity, which is prohibitively expensive for large matrices.

Selected inversion solves this by computing **only the elements we actually need** — typically those corresponding to the sparsity pattern of the original matrix. This preserves the sparse structure and dramatically reduces both memory and computation.

This operation is essential in applications like Bayesian inference, where we need diagonal elements for marginal variances, electronic structure calculations in physics, and inverse covariance estimation in statistics.

Slide 3: Visualizing Selected Inversion



Speaker Notes

To make this concrete, let me show you what selected inversion looks like for different selection patterns.

This figure shows ten cases — five for dense matrices on the left, and five for arrowhead matrices on the right.

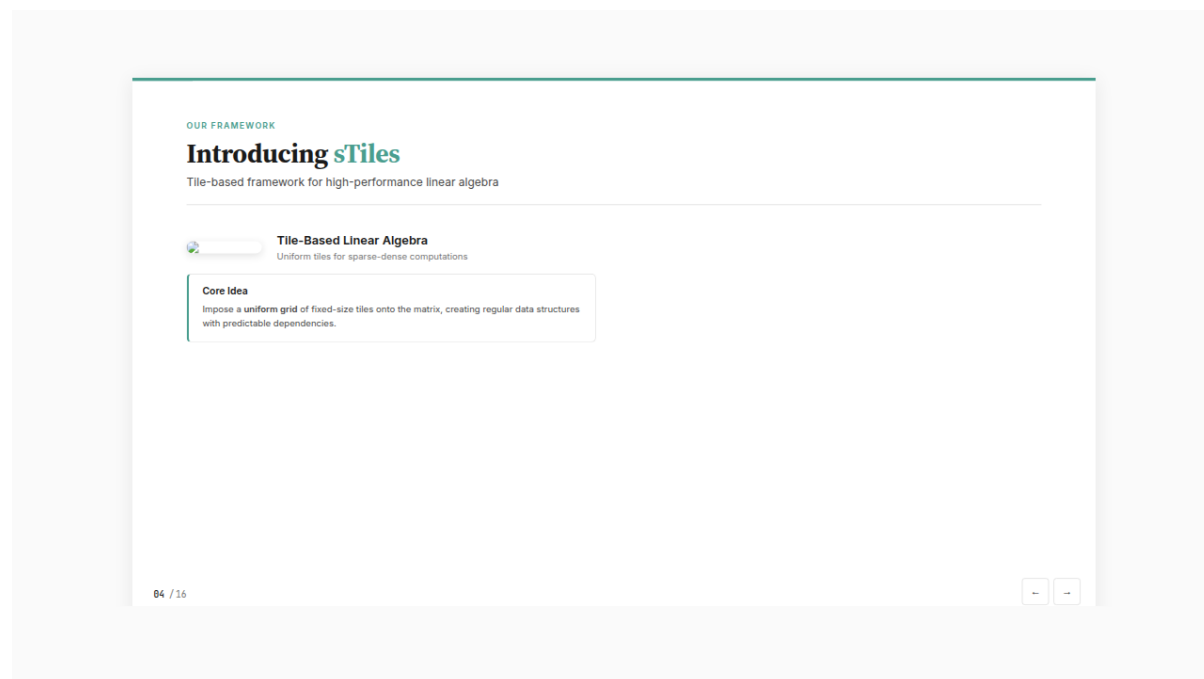
Look at cases 1 and 6: if we select the **full matrix**, we get the complete dense inverse. That's expensive.

Now look at cases 2 and 7: when we select only tiles matching the **original sparsity pattern**, something interesting happens. For the arrowhead matrix in case 7, the result **also has an arrowhead structure**. We don't need to form the full dense inverse.

Cases 4-5 and 9-10 show that if we only need off-diagonal elements, the computation is minimal.

For our work, we focus on case 7 — computing the inverse on the same pattern as the original matrix. This is the most common use case in Bayesian inference where we need the marginal variances.

Slide 4: Real-World Sparsity Patterns



Speaker Notes

So what matrices actually arise in practice?

In many real-world applications, particularly in Bayesian spatial statistics using the INLA methodology, we encounter what we call **arrowhead matrices**.

These matrices have a characteristic structure: a banded diagonal region, plus dense blocks in the last row and column — forming an arrow shape.

The key insight is that even though the original matrix is sparse, the Cholesky factor introduces **fill-in**. However, for arrowhead matrices with natural ordering, this fill-in is **predictable and structured**.

This regularity is exactly what we exploit. It allows us to use fixed-size tiles and static scheduling, which leads to excellent performance.

Slide 5: Generality of the Approach

DATA STRUCTURE

Tile Mapping

From scalar indices to contiguous tile storage

The diagram illustrates the mapping from scalar indices to contiguous tile storage. It shows a scalar (i, j) being mapped to a tile (k, n) , which is then mapped to a linear storage index. The linear storage is represented as a 2x3 grid of tiles with indices 0, 1, 2 in the first row and 3, 4, 5 in the second row. The tile at index 3 is highlighted with a blue border.

05 / 16

←
→

Speaker Notes

Before diving into our solution, I want to emphasize that while arrowhead matrices motivate our work, the algorithm is **not restricted to this structure**.

The algorithm is driven by the sparsity pattern of the Cholesky factor L , including any fill-in. For general sparse SPD matrices, the fill-induced dependency closure determines what we compute — and this closure is often **much smaller** than the full matrix.

One important note: not computing a tile doesn't mean that entry of A^{-1} is zero. It simply means we don't need it for the requested output.

Arrowhead matrices are particularly nice because natural ordering preserves structure in L , but the approach extends to any structured sparsity.

Slide 6: Introducing sTiles

CORE CONCEPT

How Selected Inversion Works

Dense Matrices (1-5)

Arrowhead Matrices (6-10)

Case 1: Full selection on dense matrix -> requires computing the entire inverse. Maximum cost.

06 / 16

Speaker Notes

Now let me introduce sTiles, our tile-based framework for high-performance linear algebra. The core idea is simple but powerful: we partition the matrix into **fixed-size dense tiles** and operate on these tiles using highly optimized BLAS and LAPACK kernels.

What makes sTiles different from supernodal approaches like PARDISO is that our tiles are **uniform in size**. This creates a regular data structure with predictable memory access patterns.

We use **static scheduling**, meaning we pre-compute exactly which tasks each core will execute before the numerical computation begins. This eliminates runtime scheduling overhead and maximizes data locality.

The framework supports both CPU parallelism via OpenMP and GPU acceleration using CUDA with cuBLAS and cuSOLVER.

Slide 7: Tile Mapping

ALGORITHM

Tile-Based Inversion

3×3 tiled matrix example: $L^T \Sigma = L^{-1}$

Σ (inverse)

Σ_{00}		
Σ_{01}	Σ_{11}	
Σ_{02}	Σ_{12}	Σ_{22}

L (Cholesky)

L_{00}	0	0
L_{01}	L_{11}	0
L_{02}	L_{12}	L_{22}

Σ (inverse)

Σ_{00}		
Σ_{01}	Σ_{11}	
Σ_{02}	Σ_{12}	Σ_{22}

Recursive equations (bottom-right → top-left):

07 / 16
← →

Speaker Notes

Let me show you how we map the matrix to tiles.

We overlay a uniform grid of tiles onto the matrix. Each tile has a fixed size — typically 120 for CPU or 600 for GPU.

Tiles that fall entirely within zero regions are simply **not stored**. We only allocate memory for tiles that contain at least one nonzero element.

This compressed tile storage gives us the benefits of dense linear algebra — predictable memory access, vectorization, cache efficiency — while still respecting the sparse structure.

The tile coordinates directly map to positions in our dependency graph, which is crucial for the parallel algorithm we'll see next.

Slide 8: The Algorithm

PARALLELIZATION
Two-Phase Parallel Algorithm
3×3 tiled matrix example

Σ (Inverse) - Parallel Execution

08 / 16

Speaker Notes

Now let's look at the algorithm itself.

Starting from the Cholesky factorization $A = LL^T$, we compute tiles of the inverse $\Sigma = A^{-1}$ using a **backward recursion**.

The computation proceeds from the bottom-right corner, working up the anti-diagonals. For a 3-by-3 tiled example, we first compute Σ_{22} from the diagonal block. Then Σ_{21} using Σ_{22} . Then Σ_{11} and Σ_{20} can be computed **in parallel** because they're on the same anti-diagonal and don't depend on each other.

This anti-diagonal parallelism is the key to our approach. As the matrix size grows, there are more tiles on each anti-diagonal, exposing more parallelism.

The operations are standard BLAS-3 kernels: TRSM for triangular solves, GEMM for matrix multiplication, LAUUM for diagonal blocks.

Slide 9: Two-Phase Parallel Algorithm

APPLICABILITY

Beyond Arrowhead Matrices

Algorithm not restricted to arrowhead structure

Why Arrowhead?

- Motivates our study and performance evaluation
- Common in INLA-based Bayesian models
- Natural ordering preserves structure in L
- Compact dependency pattern $\rightarrow$ substantial savings

09 / 16

Speaker Notes

We organize the computation into two phases for maximum parallelism.

Phase 1 computes all diagonal blocks Σ_{ii} in parallel. These only depend on the corresponding L_{ii} blocks, so they're completely independent.

Phase 2 handles the off-diagonal tiles and the updates to diagonal tiles. Here we exploit the anti-diagonal parallelism I mentioned. Tiles on the same anti-diagonal can be computed concurrently.

The step indicator shows the progression: Phase 1 handles all diagonals simultaneously, then Phase 2 proceeds level by level through the anti-diagonals.

[Advance to final step]

And here's the crucial optimization: we **skip zero tiles entirely**. For arrowhead matrices, this means we never even look at the zero blocks in the middle.

Tasks are **pre-assigned to cores** at the start. Each core has its task list computed once, with no runtime scheduling overhead. This static approach is what gives us the performance advantage.

Slide 10: Test Matrices

EXPERIMENTAL SETUP

Test Matrices

Arrowhead structured matrices from INLA-based models

ID	Size	Bandwidth	Arrow	Density
Small (10K)				
1-3	10,010	100-300	10	0.4-0.6%
4-6	10,200	100-300	200	3.9-4.1%
Medium (100K)				
7-9	100,010	1K-3K	10	0.1-0.3%
10-12	100,200	1K-3K	200	0.5-0.6%
Large (500K)				
13-15	500,010	1K-3K	10	0.02-0.05%
16-18	500,200	1K-3K	200	0.1-0.13%

Matrix Structure

Arrowhead: Block diagonal + dense last row/column

Bandwidth: Width of diagonal band

Arrow thickness: Size of arrowhead blocks

10 / 16

Speaker Notes

For our experimental evaluation, we use test matrices that reflect real INLA-based Bayesian models.

The matrices are organized into three groups by size: 10K, 100K, and 500K rows.

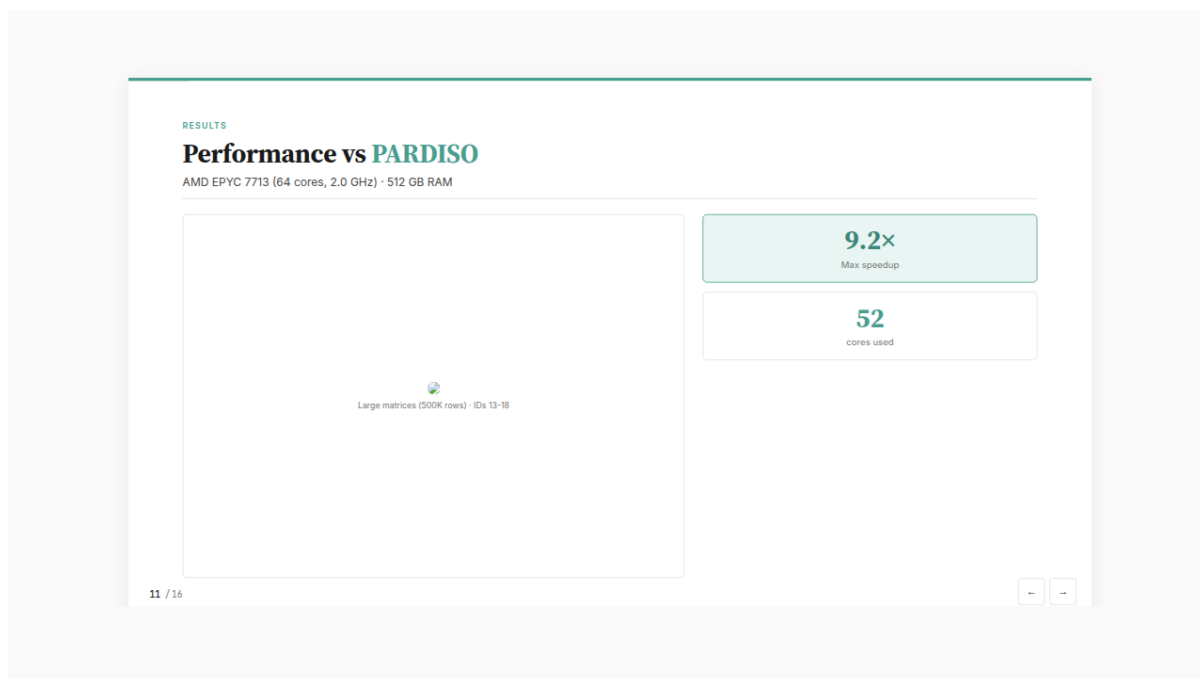
Within each group, we vary the bandwidth and arrowhead thickness. The bandwidth controls the width of the diagonal band, while the arrowhead thickness determines the size of the dense blocks at the end.

Matrix IDs 1 through 18 cover a range of configurations, from small problems with narrow bandwidth to large problems with wide bandwidth.

For the GPU experiments, we use a separate high-intensity test case: a 50K matrix with bandwidth 15,000. This ensures enough computational work to fully utilize the A100 GPU.

These matrices are representative of what you'd encounter in spatial statistics applications with varying numbers of spatial locations and time points.

Slide 11: Performance vs PARDISO



Speaker Notes

Now for the results. We compare sTiles against Panua-PARDISO, which is the state-of-the-art solver for this class of problems.

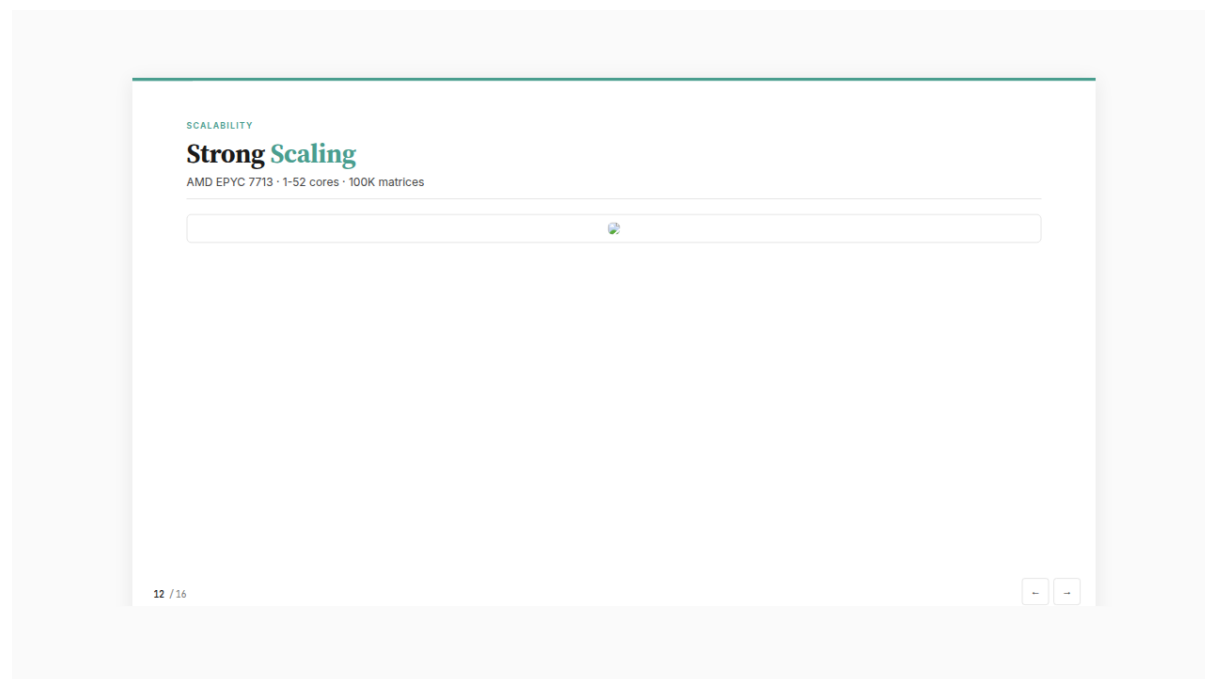
This plot shows execution time for the large 500K matrices on a 64-core AMD EPYC system.

The results are striking: sTiles achieves up to **9.2 times speedup** over PARDISO.

Why is sTiles faster? The uniform tile structure enables better cache reuse. Memory access patterns are predictable, which helps the hardware prefetcher. And our static scheduling eliminates the overhead of dynamic task management.

The performance gap is largest for high-bandwidth matrices where there's more computational work and more opportunity for parallelism.

Slide 12: Scalability



Speaker Notes

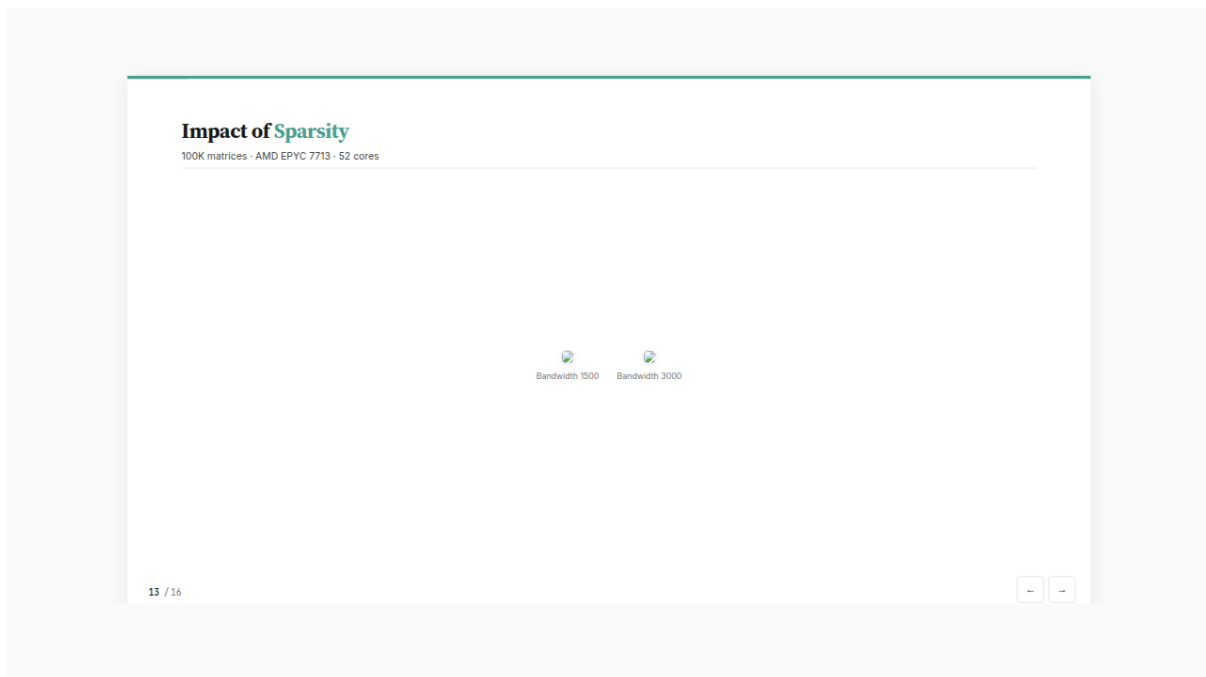
This plot shows strong scaling behavior as we increase the core count from 1 to 52. For Matrix ID 12, which has high bandwidth, sTiles achieves **16 times speedup** at 52 cores.

Notice the difference in scaling behavior: sTiles continues to improve as we add cores, while PARDISO's performance **saturates early**.

This is the benefit of our static scheduling approach. With dynamic scheduling, there's increasing overhead from thread synchronization and task stealing as core counts grow. Our pre-computed task assignment avoids this entirely.

The speedup over PARDISO actually **increases** with more cores, because our approach scales better.

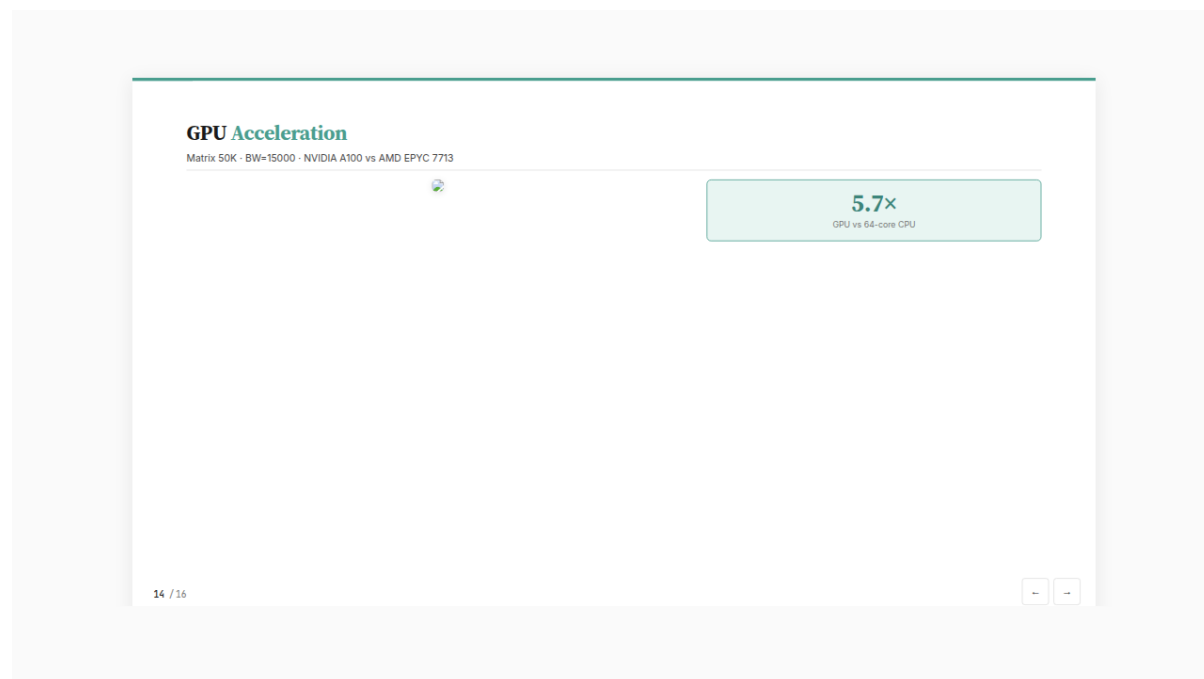
Slide 13: Sparsity Impact



Speaker Notes

This experiment examines how performance changes as matrix density increases. We use 100K matrices with two different bandwidths: 1500 on the left, 3000 on the right. Within each set, we progressively increase the density of the arrowhead band. The key observation is that sTiles maintains **remarkably stable performance** across all density levels. The execution time is nearly flat. PARDISO, on the other hand, shows significant performance variation. Its time increases substantially as density grows. At the highest density levels, sTiles is up to **12 times faster** than PARDISO. This stability comes from our tile-based approach: once a tile is allocated, we process it the same way regardless of how full it is. PARDISO's supernodal approach is more sensitive to the actual nonzero structure.

Slide 14: GPU Acceleration



Speaker Notes

Finally, let's look at GPU acceleration.

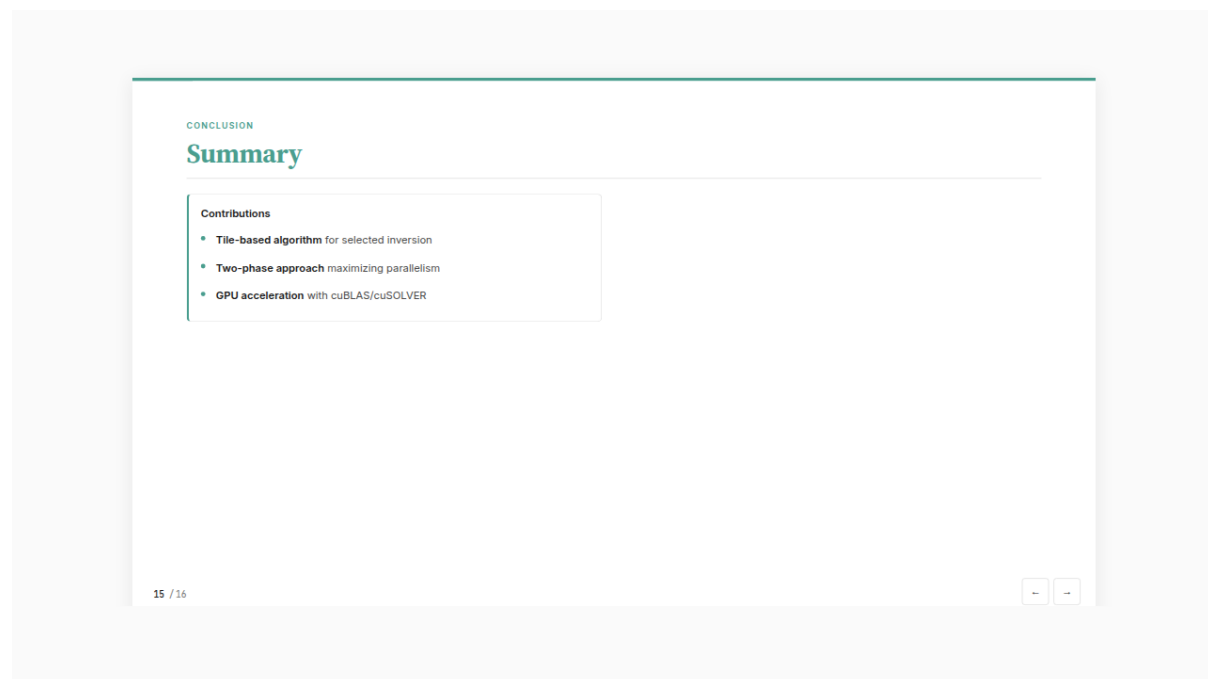
For the GPU experiments, we use a large matrix with bandwidth 15,000 to ensure sufficient computational intensity.

The GPU implementation uses **larger tiles** — 600 instead of 120 — because GPU kernels need more work per invocation to be efficient. Each CPU thread manages its own CUDA stream for concurrent kernel execution.

The results show the A100 GPU achieving **5.7 times speedup** over the 64-core CPU. There's an interesting trade-off here: larger tiles maximize kernel throughput but reduce the number of parallel tasks in the dependency graph. This limits scalability with multiple streams.

The best GPU time of 3.5 seconds with 32 streams is achieved when we balance these factors. This validates that sTiles can effectively exploit GPU acceleration for large, computationally intensive problems.

Slide 15: Summary



Speaker Notes

Let me summarize our contributions.

We presented a **tile-based algorithm** for selected inversion that exploits the regular structure of arrowhead matrices while remaining applicable to general sparse SPD matrices.

Our **two-phase approach** maximizes parallelism through anti-diagonal scheduling and static task assignment.

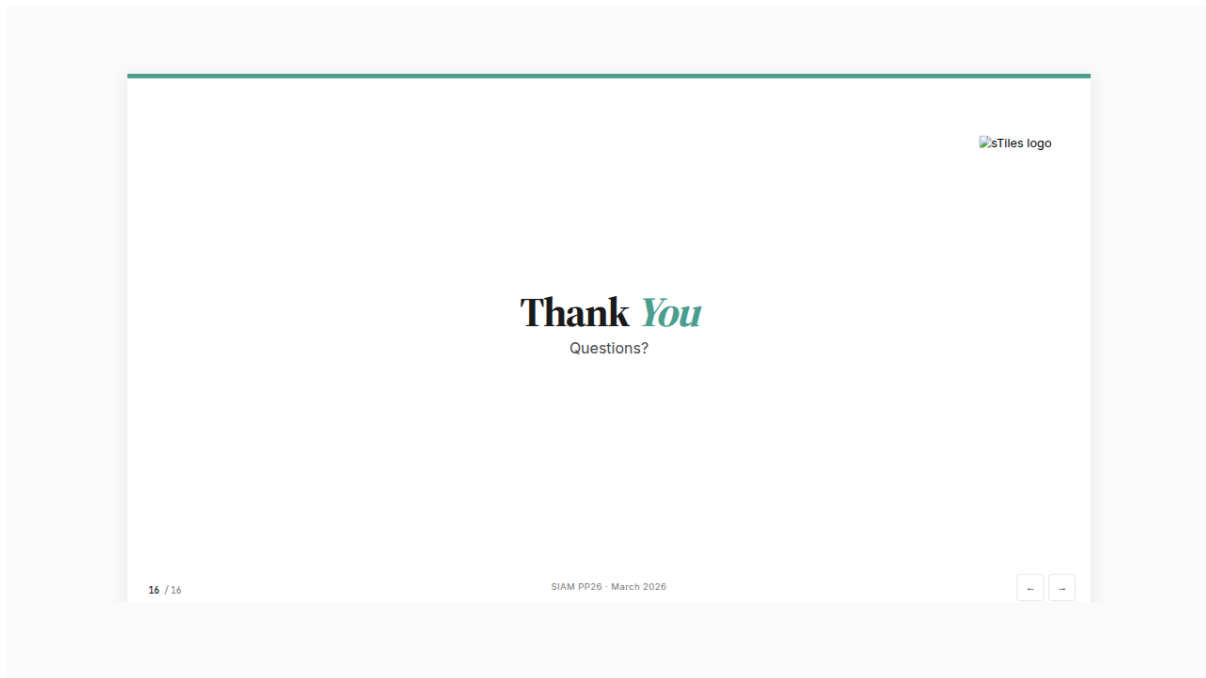
The **GPU implementation** with cuBLAS and cuSOLVER extends the approach to massively parallel hardware.

In terms of results: we achieve up to **9 times speedup** over PARDISO on CPU, and an additional **5.7 times speedup** on GPU. Performance remains stable across varying sparsity levels.

For future work, we're exploring multi-GPU support for even larger problems and distributed memory implementations for cluster-scale computations.

The practical impact is enabling faster Bayesian inference for large-scale spatial statistics, where selected inversion is often the computational bottleneck.

Slide 16: Thank You



Speaker Notes

Thank you for your attention.
I'm happy to take any questions about the algorithm, implementation, or results.

End of presentation script